

Function parameters and arguments

(CC-BY: Maria Skeppstedt)

Fikonspråket

```
1 def code_word(word):
2     vowels = ("a", "e", "i", "o", "u", "y") # No Swedish
3
4     vow_idx = [word.index(v) for v in vowels if v in word]
5     split_at = min(vow_idx) + 1 # split after first vowel
6     first, second = word[:split_at], word[split_at:]
7     return "fi" + second, first + "kon"
8
9 print(code_word("coffee"))
10 print(code_word("stump"))
11 print(code_word("velociped"))
```

('fiffee', 'cokon')

('fimp', 'stukon')

('filociped', 'vekon')

The result is the word transformed to “fikonspråket”.

```
1 def code_word(word):
2     vowels = ("a", "e", "i", "o", "u", "y") # No Swedish
3
4     vow_idx = [word.index(v) for v in vowels if v in word]
5     split_at = min(vow_idx) + 1 # split after first vowel
6     first, second = word[:split_at], word[split_at:]
7     return "fi" + second, first + "kon"
8
9 res = code_word("stump")
10 print(res)
```

(‘fimp’, ‘stukon’)

Note how easy it is in Python to return several values. They are automatically transformed into a tuple.

```
1 print(type(res))
```

<class 'tuple'>

The tuple can also be directly assigned to variables.

```
1 def code_word(word):
2     vowels = ("a", "e", "i", "o", "u", "y") # No Swedish
3
4     vow_idx = [word.index(v) for v in vowels if v in word]
5     split_at = min(vow_idx) + 1 # split after first vowel
6     first, second = word[:split_at], word[split_at:]
7     return "fi" + second, first + "kon"
8
9 first_word, second_word = code_word("stump")
10 print(first_word)
11 print(second_word)
```

```
'fimp'
'stukon'
```

```
1 print(type(res))
```

Or more generally, variables can be assigned from a tuple or list (and all other iterables):

```
1 a, b, c = [1, 2, 3]
2 print(a)
3 print(b)
4 print(c)
```

1
2
3

When the number of list elements don't match the number of variables, it will not work. E.g.,
`a, b, c = [1, 2, 3, 4],`
will not work.

Parameters

Parameters are the names in a function definition.

This function has one parameter: word

```
1 def code_word(word):
2     vowels = ("a", "e", "i", "o", "u", "y") # No Swedish
3
4     vow_idx = [word.index(v) for v in vowels if v in word]
5     split_at = min(vow_idx) + 1 # split after first vowel
6     first, second = word[:split_at], word[split_at:]
7     return "fi" + second, first + "kon"
8
9 print(code_word("coffee"))
10 print(code_word("stump"))
11 print(code_word("velociped"))
```

Arguments

Arguments are the values that are passed to a function when it is called.

In this case, the function is called with the arguments “coffee”, “stump” and “velociped”

```
1 def code_word(word):
2     vowels = ("a", "e", "i", "o", "u", "y") # No Swedish
3
4     vow_idx = [word.index(v) for v in vowels if v in word]
5     split_at = min(vow_idx) + 1 # split after first vowel
6     first, second = word[:split_at], word[split_at:]
7     return "fi" + second, first + "kon"
8
9 print(code_word("coffee"))
10 print(code_word("stump"))
11 print(code_word("velociped"))
```

Default parameter values

Very practical, to achieve flexibility for the users of your code who want that, and simplicity for those who just want to use the default.

```
1 def code_word(word, vowels = ("a", "e", "i", "o", "u", "y")):
2
3     vow_idx = [word.index(v) for v in vowels if v in word]
4     split_at = min(vow_idx) + 1 # split after first vowel
5     first, second = word[:split_at], word[split_at:]
6     return "fi" + second, first + "kon"
7
8 print(code_word("coffee"))
9 print(code_word("stump"))
10 print(code_word("velociped", ("a", "e")))
```

Default parameter values are used a lot in scikit-learn to achieve flexibility and simplicity

LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='deprecated', *,  
C=1.0, l1_ratio=0.0, dual=False, tol=0.0001, fit_intercept=True,  
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs',  
max_iter=100, verbose=0, warm_start=False, n_jobs=None) \[source\]
```

Logistic Regression (aka logit, MaxEnt) classifier.

There are many possible, different parameters when creating a Logistic Regression instance. With this, a lot of flexibility is achieved for the user.

But, as a start, the user can create a classifier, calling the constructor without any arguments at all ...

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Training the model
4 classes = ("HARMLESS", "DANGEROUS")
5 features = ("Hungerlevel", "Evilness", "Strength")
6 clf = LogisticRegression()
7 X = [[0.9, 0.1, 0.3],
8      [0.9, 0.9, 0.7],
9      [0.8, 0.2, 0.5],
10     [0.3, 0.8, 0.7]]
11 y = [0, 1, 0, 1]
12 clf.fit(X, y)
13
14 # Using the model
15 creatues = [("trolla", [0.3, 0.9, 0.6]),
16            ("bilbo", [0.8, 0.2, 0.3])]
17 preds = clf.predict([x for _, x in creatues])
18 names = [name for name, _ in creatues]
19 for name, y in zip(names, preds):
20     print(name, ":", classes[y])
```

trolla : DANGEROUS

bilbo : HARMLESS

Keyword arguments

You can name your arguments, i.e. use Keyword arguments.

In the second example case below, the last argument is a keyword argument. In the third example case, both arguments are keyword arguments.

```
1 def code_word(word, vowels = ("a", "e", "i", "o", "u", "y")):
2     vow_idx = [word.index(v) for v in vowels if v in word]
3     split_at = min(vow_idx) + 1 # split after first vowel
4     first, second = word[:split_at], word[split_at:]
5     return "fi" + second, first + "kon"
6
7 print(code_word("stump"))
8 print(code_word("velociped", vowels = ("a", "e")))
9 print(code_word(word="coffee", vowels = ("a", "e", "i", "o")))
```

Note that once you have started using keyword arguments in your argument list, you cannot go back to using non-keyword arguments later in the list. This does **not** work:

```
code_word(word="stump", ("a", "e"))
```

To use keyword arguments can sometimes make the code easier to understand. (And can lead to errors being avoided.)

```
1 def code_word(word, lower_case=True, start_from_back=False):
2     vowels = ("a", "e", "i", "o", "u", "y")
3
4     if lower_case:
5         word = word.lower()
6     else:
7         vowels = [el.upper() for el in vowels] + list(vowels)
8
9     vow_idx = [word.index(v) for v in vowels if v in word]
10    if start_from_back:
11        split_at = max(vow_idx) - 1 # before first vowel
12    else:
13        split_at = min(vow_idx) + 1 # after first vowel
14    first, second = word[:split_at], word[split_at:]
15    return "fi" + second, first + "kon"
```

Which is easiest to understand?

```
1 print(code_word("VElociped", False, True))
2 print(code_word("VElociped", lower_case=False, \
3     start_from_back=True))
```

Variable-length arguments:

When you want to create a function that can be called with any number of arguments.

Add a star to show it is a variable length argument.

By convention, a variable-length argument is named `arg`.

```
1 def acc_sum(*arg):
2     a_sum = 0
3     for el in arg:
4         a_sum = a_sum + a_sum + el
5     return a_sum
6
7 print(acc_sum(2))
8 print(acc_sum(1, 20))
9 print(acc_sum(1, 20, 100))
```

2

22

144

Variable-length arguments:

The `arg` argument becomes a tuple. So the code in the function can just treat it as a tuple

```
1 def acc_sum(*arg):
2     print("Tuple?", isinstance(arg, tuple))
3     a_sum = 0
4     for el in arg:
5         a_sum = a_sum + a_sum + el
6     return a_sum
7
8 print(acc_sum(2))
9 print(acc_sum(1, 20))
10 print(acc_sum(1, 20, 100))
```

Tuple? True

2

Tuple? True

22

Tuple? True

144

Unpack a list

```
1 def len_difference(word_1, word_2):  
2     return abs(len(word_1) - len(word_2))  
3  
4 print(len_difference("123", "12345"))
```

What if you have a list, and want to send the elements as individual arguments to a function?

```
1 existing_list = ["123", "12345"]
```

This will **not** work: `len_difference(existing_list)`
since you are only sending one argument to a function that expects two arguments

But if you unpack the list first (with a `*`), it will work:

```
1 print(len_difference(*existing_list))
```

Variable-length keyword arguments:

In a similar fashion, you can have variable-length keyword arguments. You indicate that by: `**`

By convention, this argument is named `kwargs`.

```
1 def print_creature_info (**kwargs):  
2     for key, value in kwargs.items():  
3         print (f"{key}\t{value}")  
4  
5 print_creature_info (name="Bilbo", age=300, hunger_level=0.3)
```

name Bilbo

age 300

hunger_level 0.3

Variable-length keyword arguments

The `kwargs` argument becomes a dictionary. So the code in the function can just treat it as a dictionary.

```
1 def print_creature_info(**kwargs):  
2     print("Type:", type(kwargs))  
3     #for key, value in kwargs.items():  
4         #print(f"{key}\t{value}")
```

Type: <class 'dict'>

Combining *args and **kwargs

You can also have both in the same function.

```
1 def print_creature_info(*args, **kwargs):
2     for key, value in kwargs.items():
3         print(f"{key}\t{value}")
4     print("The creature carries the following items: ")
5     for el in args:
6         print(el)
7
8 print_creature_info("ring", "pipe", "breakfast", \
9     name="Bilbo", age=300, hunger_level=0.3)
```

name Bilbo

age 300

hunger_level 0.3

The creature carries the following items:

ring

pipe

breakfast

Combining with a normal parameter

```
1 def print_creature_info(name, *args, **kwargs):
2     print(f"{name.upper()}: ")
3     for key, value in kwargs.items():
4         print(f"{key}\t{value}")
5     print(f"{name} carries the following items: ")
6     for el in args:
7         print(el)
8
9 print_creature_info("Bilbo", "ring", "pipe", "breakfast",\
10                    age=300, hunger_level=0.3)
```

BILBO:

age 300

hunger_level 0.3

Bilbo carries the following items:

ring

pipe

breakfast

Combining with a normal parameter

The different argument types need to be in this order, otherwise, it will not work.

```
1 def print_creature_info(name, *args, **kwargs):
2     print(f"{name.upper()}: ")
3     for key, value in kwargs.items():
4         print(f"{key}\t{value}")
5     print(f"{name} carries the following items: ")
6     for el in args:
7         print(el)
8
9 print_creature_info("Bilbo", "ring", "pipe", "breakfast", \
10                    age=300, hunger_level=0.3)
```

A method for achieving flexibility while hiding complexity

To use variable-length keyword arguments in your functions is yet a method for achieving flexibility while hiding complexity.

```
1 def print_creature_info(name, lives_in="Middle-Earth",\
2     **kwargs):
3     print(f"{name} lives in {lives_in}")
4     if kwargs:
5         print(f"{name} has the following attributes")
6         for key, value in kwargs.items():
7             print(f"{key}\t{value}")
```

1. For the user who just wants the default behaviour:

```
1 print_creature_info("Bilbo")
```

Bilbo lives in Middle-Earth

A method for achieving flexibility while hiding complexity

To use variable-length keyword arguments in your functions is yet a method for achieving flexibility while hiding complexity.

```
1 def print_creature_info(name, lives_in="Middle-Earth",\
2     **kwargs):
3     print(f"{name} lives in {lives_in}")
4     if kwargs:
5         print(f"{name} has the following attributes")
6         for key, value in kwargs.items():
7             print(f"{key}\t{value}")
```

2. For the user who wants a little more flexibility, but not that much complexity:

```
1 print_creature_info("Bilbo", lives_in="Dortmund")
```

Bilbo lives in Dortmund

A method for achieving flexibility while hiding complexity

To use variable-length keyword arguments in your functions is yet a method for achieving flexibility while hiding complexity.

```
1 def print_creature_info(name, lives_in="Middle-Earth", \
2     **kwargs):
3     print(f"{name} lives in {lives_in}")
4     if kwargs:
5         print(f"{name} has the following attributes")
6         for key, value in kwargs.items():
7             print(f"{key}\t{value}")
```

3. For the user who wants a lot of flexibility: (in a real setting the keywords would be used for something more exciting than just being printed.)

```
1 print_creature_info("Bilbo", age=300, \
2     hunger_level=0.3, lives_in="Dortmund")
```

Bilbo lives in Dortmund

Bilbo has the following attributes

age 300

hunger_level 0.3

Matplotlib uses ****kwargs** a lot:

matplotlib.pyplot.scatter

```
matplotlib.pyplot.scatter(x, y, s=None, c=None, *, marker=None,  
cmap=None, norm=None, vmin=None, vmax=None, alpha=None, linewidths=None,  
edgecolors=None, colorizer=None, plotnonfinite=False, data=None, **kwargs)
```

[\[source\]](#)

A scatter plot of y vs. x with varying marker size and/or color.

Parameters:

x, y : *float or array-like, shape (n,)*

The data positions.

s : *float or array-like, shape (n,), optional*

The marker size in points**2 (typographic points are 1/72 in.). Default is

```
rcParams['lines.markersize'] ** 2.
```

1. Simplest use: you can create a scatter plot by just providing the x and y coordinates for the data.
2. More advanced: You can provide more detailed instructions for the plot, by using other than the default arguments.
3. Most advanced: you can provide detailed instructions for what your plot should look like using the variable-length keyword arguments.

Let's say we want to plot the data that we used for training our logistic regression model.

```
1 # Training the model
2 classes = ("HARMLESS", "DANGEROUS")
3 features = ("Hungerlevel", "Evilness", "Strength")
4 clf = LogisticRegression()
5 X = [[0.9, 0.1, 0.3],
6      [0.9, 0.9, 0.7],
7      [0.8, 0.2, 0.5],
8      [0.3, 0.8, 0.7]]
9 y = [0, 1, 0, 1]
10 clf.fit(X, y)
11
12 # Using the model
13 creatues = [("trolla", [0.3, 0.9, 0.6]),
14             ("bilbo", [0.8, 0.2, 0.3])]
15 preds = clf.predict([x for _, x in creatues])
16 names = [name for name, _ in creatues]
17 for name, y in zip(names, preds):
18     print(name, ":", classes[y])
```

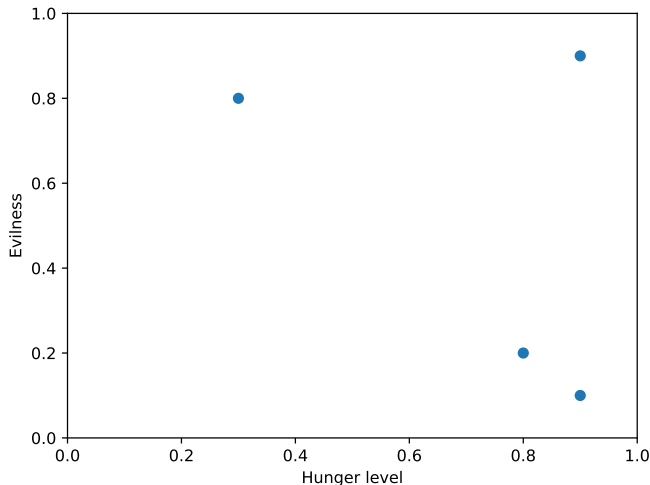
1. Most basic plot

(hungerlevel and evilness are plotted)

```
1 features = ("Hunger level", "Evilness", "Strength")
2 X = np.array([[0.9, 0.1, 0.3],
3               [0.9, 0.9, 0.7],
4               [0.8, 0.2, 0.5],
5               [0.3, 0.8, 0.7]])
6 # Plot hunger_level, Evilness
7 plt.scatter(X[:,0], X[:,1])
8 plt.xlabel(features[0])
9 plt.ylabel(features[1])
10 plt.ylim(0,1)
11 plt.xlim(0,1)
12 plt.savefig('simplest.pdf')
```

1. Most basic plot

Hunger level and Evilness for our four data points in the training data. We don't care about detailed settings, and only provide x and y values.



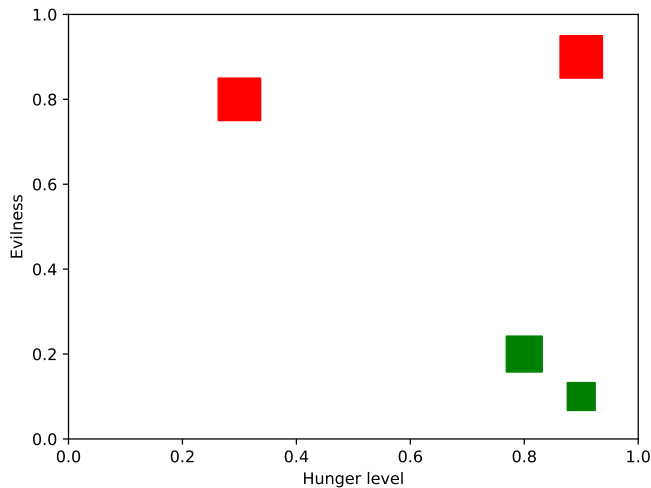
2. More settings

Using other than the default values for some of the standard parameters. (Size, marker type and colour.)

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 colors = ["green", "red"]
5 features = ("Hunger level", "Evilness", "Strength")
6 X = np.array([[0.9, 0.1, 0.3],
7               [0.9, 0.9, 0.7],
8               [0.8, 0.2, 0.5],
9               [0.3, 0.8, 0.7]])
10 y = [0, 1, 0, 1]
11
12 plt.scatter(X[:,0], X[:,1],\
13             s = [el*1000 for el in X[:,2]],
14             marker = 's',
15             c = [colors[el] for el in y])
16 plt.xlabel(features[0]); plt.ylabel(features[1])
17 plt.ylim(0,1); plt.xlim(0,1)
18 plt.savefig('more-settings.pdf')
```

2. More settings

Using other than the default values for some of the standard parameters. (Size, marker type and colour.)



3. Using ****kwargs**

The ****kwargs** are at the end of the documentation for scatter. (Only to read for those who want more advanced settings.)

****kwargs** : [PathCollection](#) *properties*

Property	Description
agg_filter	a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array and two offsets from the bottom left corner of the image
alpha	array-like or float or None
animated	bool
antialiased or aa or antialiaseds	bool or list of bools

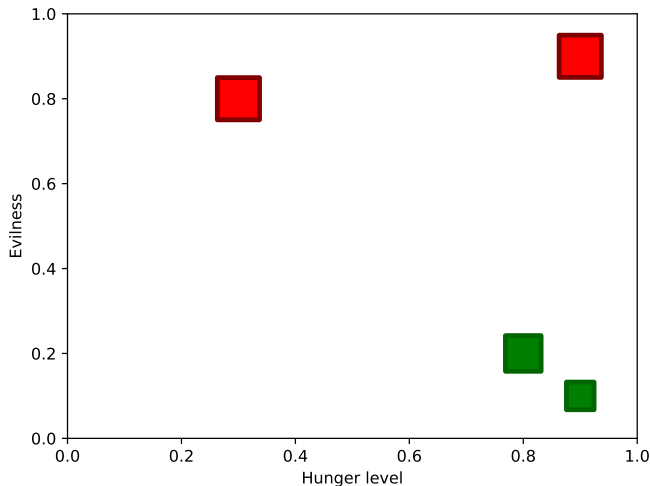
3. Using **kwargs

Also using parameters “hidden away” among possible kwargs (marker line colour and width).

```
1 colors = ["green", "red"]
2 l_c = ["darkgreen", "maroon"]
3 features = ("Hunger level", "Evilness", "Strength")
4 X = np.array([[0.9, 0.1, 0.3],
5               [0.9, 0.9, 0.7],
6               [0.8, 0.2, 0.5],
7               [0.3, 0.8, 0.7]])
8 y = [0, 1, 0, 1]
9 # Plot hunger_level, Evilness
10 plt.scatter(X[:,0], X[:,1],\
11             s = [el*1000 for el in X[:,2]],
12             marker = 's',
13             c = [colors[el] for el in y],
14             edgecolor = [l_c[el] for el in y],
15             linewidth = 3)
16 plt.xlabel(features[0]); plt.ylabel(features[1])
17 plt.ylim(0,1); plt.xlim(0,1)
18 plt.savefig('kwarg-settings.pdf')
```

3. Using ****kwargs**

Also using parameters “hidden away” among possible kwargs (marker line colour and width).



Using a dictionary as keyword arguments

If you have a dictionary with keys that match the keyword arguments of a function, you can write `**` to use the values in the dictionary as keyword arguments.

```
1 def len_difference(word_1, word_2):  
2     return abs(len(word_1) - len(word_2))  
3  
4 print(len_difference(word_1="123", word_2="12345"))  
5  
6 d = {"word_1": "123", "word_2": "12345"}  
7 print(len_difference(**d)) # Same result as above
```

2

2

Methods

All this, of course, also apply to methods. But, as usual, you need to start the parameters with `self`

```
1 class CreaturePrinter:
2     def print_creature_info(self, name, *args, **kwargs):
3         print(f"{name.upper()}: ")
4         for key, value in kwargs.items():
5             print(f"{key}\t{value}")
6         print(f"{name} carries the following items: ")
7         for el in args:
8             print(el)
9
10 printer = CreaturePrinter()
11 printer.print_creature_info("Bilbo", "ring", "pipe", \
12     "breakfast", age=300, hunger_level=0.3)
```

```
BILBO:
age 300
hunger_level 0.3
Bilbo carries the following items:
ring
pipe
breakfast
```

(or actually, you can technically name it anything. Python doesn't care, as long as you provide this extra first parameter in your method definition. But you will cause confusion:)

```
1 class CreaturePrinter:
2     def print_creature_info(BREAKING_ALL_CONVENTION, \
3         name, *args, **kwargs):
4         print(f"{name.upper()}: ")
5         for key, value in kwargs.items():
6             print(f"{key}\t{value}")
7         print(f"{name} carries the following items: ")
8         for el in args:
9             print(el)
10
11 printer = CreaturePrinter()
12 printer.print_creature_info("Bilbo", "ring", "pipe", \
13     "breakfast", age=300, hunger_level=0.3)
```

```
BILBO:
age 300
hunger_level 0.3
Bilbo carries the following items:
ring
pipe
breakfast
```

If you forget `self` and use variable length arguments, you will get confusing error messages.

```
1 class CreaturePrinter:
2     # Here "self" is forgotten:
3     def print_creature_info(name, *args, **kwargs):
4         print(f"{name.upper()}:")
5         for key, value in kwargs.items():
6             print(f"{key}\t{value}")
7         print(f"{name} carries the following items: ")
8         for el in args:
9             print(el)
10
11 printer = CreaturePrinter()
12 printer.print_creature_info("Bilbo", "ring", "pipe", \
13     "breakfast", age=300, hunger_level=0.3)
```

```
print(f"{name.upper()}:")
    ~~~~~
```

AttributeError: 'CreaturePrinter' object has no attribute 'upper'